INTERVIEW STUDY GUIDE · VOLUME 2

Firmware and Platform Deep Dive

How the current implementation works from Zephyr discovery and boot through clocking, UART, model execution, interrupts, watchdog recovery, and host validation.

AM13E230 TINIElite Edge AI · Jainil Bavishi · 23 October 2026

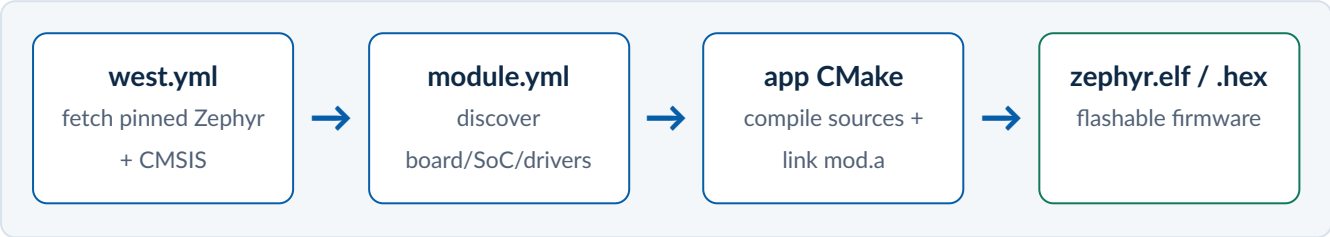
Contents

1. How the firmware is assembled
2. Devicetree, bindings, and configuration
3. Boot and 200 MHz clock configuration
4. Pin control and UART driver
5. Protocol-v2 design
6. Python host client
7. Generated model and binary ABI
8. TINIElite driver and memory interface
9. Interrupt-driven inference
10. Timing measurements
11. Timeouts, reset, and watchdog
12. Application state machine and concurrency
13. File-by-file responsibility map

1. How the firmware is assembled

The repository is a Zephyr module and a west manifest repository. These two ideas solve different problems:

- **west manifest:** `west.yml` pins the upstream Zephyr commit and imports only required modules such as CMSIS. This makes dependency revisions explicit.
- **Zephyr module:** `zephyr/module.yml` tells Zephyr that this repository contributes CMake/Kconfig content and is also a board, devicetree, and SoC root.



1.1 Why pin Zephyr?

Zephyr APIs, build rules, bindings, and board conventions evolve. A commit hash makes the build deterministic and prevents an unrelated upstream change from silently altering the platform. The current pin is `9574b65fbaa01aa0c8f418bd7a6b97beb15fcc54`.

1.2 CMake linking details

`app/CMakeLists.txt` compiles the six application source files, exposes the generated model header, and links `models/waveform_classifier/artifacts/mod.a`. The option `-Wl,-u,tvmgen_default_run` forces the archive entry point to be pulled into the ELF even if archive extraction rules would otherwise discard it.

The generated library is statically linked. Replacing the model therefore means supplying a compatible archive/header and rebuilding the firmware; this is not a dynamic model loader.

1.3 Source code, object code, library, ELF, and HEX

This is the build chain behind the previous paragraph. C source files are human-readable instructions. The Arm compiler translates each source file into an object file (`.o`): machine code plus data, with unresolved names allowed. A static archive (`.a`) is a collection of such object files. The linker resolves the names across Zephyr, the application, drivers, and `mod.a`, assigns final flash/RAM addresses, and emits `zephyr.elf`. A HEX conversion then creates the addressable image that DSLite programs to flash.

```
C source + generated header
    -> compiler -> .o object files
    -> linker + Zephyr/project objects + mod.a -> zephyr.elf
    -> objcopy/converter -> zephyr.hex -> XDS110/DSLite -> board flash
```

Object code is therefore not “source code in another format.” It is the processor-specific binary translation of source. It can be inspected through symbols, but it is not a complete runnable firmware until the linker

resolves all required functions and memory locations. The CI symbol gate checks the finished ELF precisely because a model archive merely existing on disk does not prove it was pulled into the firmware.

1.4 Kconfig versus devicetree

Mechanism	Question it answers	Example
Kconfig	Which software features are compiled?	`CONFIG_TINYENGINE_AM13E230X`, stack size, serial, reboot, test-only fault injection.
Devicetree	Which hardware exists and where/how is it connected?	UART base addresses, NPU IRQ 41, watchdog address, clock rates, pins.
CMake	Which source/object files are compiled and linked?	Add drivers, application sources, and `mod.a`.

Kconfig is compile-time feature selection. Kconfig files define available symbols and dependencies; board defaults and `app/prj.conf` request values; the configuration system produces the final `.config` used for compilation. It is not a runtime setting that a UART command changes after flashing. The NPU/watchdog fault-injection symbols are a concrete example: when enabled, diagnostic protocol messages exist so recovery can be tested; a deployment configuration disables them.

Devicetree is hardware description. NPU IRQ 41, UART addresses, PA0/PA1 pin routing, and watchdog base addresses are physical facts—not user preferences. Devicetree communicates those facts to a driver. CMake is a third, separate concern: it chooses sources/libraries to compile and link. For a longer teaching explanation and diagrams, see Volume 0.

2. Devicetree, bindings, and configuration

2.1 SoC description

`dts/arm/ti/am13e23019.dtsi` describes the Cortex-M33, flash and SRAM regions, fixed clock nodes, SYSCTL, pin controller, flash controller, UART, TINIElite, and WWDT0. Disabled-by-default peripheral nodes become usable when the board/application sets `status = "okay"`.

2.2 Board description

`lp_am13e230.dts` identifies the board, chooses the flash/SRAM/console, declares 200 MHz CPU metadata, assigns the 100 MHz UART source, and enables UART. `app.overlay` enables TINIElite and WWDT0 specifically for this application.

2.3 Why binding YAML files matter

A binding defines the schema for a compatible device. It documents expected properties and lets Zephyr validate the devicetree at build time. For example, the UART binding requires register regions, register names, a clock, current speed, and pinctrl support. A typo or missing required property becomes a build error rather than a mysterious runtime failure.

2.4 Compile-time device construction

Macros such as `DEVICE_DT_INST_DEFINE`, `DT_INST_REG_ADDR_BY_NAME`, and `DT_INST_IRQN` turn validated devicetree nodes into constant driver configuration. This removes runtime device enumeration and catches many integration errors during compilation.

3. Boot and 200 MHz clock configuration

> **CRITICAL LESSON**
> Declaring `clock-frequency = <200000000>` does not configure hardware. It only tells software what the frequency is supposed to be. An early version declared 200 MHz while the chip actually ran at 32 MHz, making cycle-to-time conversions wrong.

3.1 Early initialization

`soc/ti/am13x/clock.c` registers `am13e230_clock_init` with `SYS_INIT(..., PRE_KERNEL_1, 0)`. It runs before the Zephyr kernel and timer depend on the declared clock. The sequence follows the retained TI SDK configuration:

1. Execute a RAM-resident function that disables flash cache paths temporarily and programs three flash read wait states.
2. Configure the dedicated X1/X2 pins and start the board's 25 MHz crystal.
3. Wait until hardware reports the high-frequency source is good.
4. Disable the PLL and wait for its off status before changing its configuration.
5. Select the 25 MHz reference, predivide by 2, multiply by 32, and divide the output by 2: `(25 MHz / 2 × 32) / 2 = 200 MHz`.
6. Load device-specific PLL analog parameters from retained addresses.
7. Enable the PLL and wait for lock/good status.
8. Switch MCLK gradually through external division by 4, division by 2, then bypass to avoid an abrupt frequency transition.
9. Set the MCLK divider configuration required by the TI sequence.

3.2 Why configure flash from RAM?

At higher CPU frequencies, flash cannot return instructions in a single core cycle. Wait states give flash extra cycles. Modifying flash timing while executing code from that same flash is risky, so the small routine is marked `__ramfunc` and runs from SRAM.

3.3 Bounded status waits

Crystal and PLL status loops use a finite iteration count and return `-ETIMEDOUT` if hardware never becomes healthy. A boot failure is preferable to silently continuing with inconsistent timing assumptions.

3.4 Independent read-back

`app_clock.c` reads raw clock and flash registers, derives active MCLK from divisors, and returns twelve words to the host. Five validation bits cover source selection, high-speed clock health, PLL/reference health, expected frequency, and sufficient flash wait states. Hardware evidence recorded all five bits set, 200 MHz MCLK, 100 MHz UART source, and three wait states.

4. Pin control and UART driver

4.1 Pin routing

A physical package pin can serve many peripheral functions. The board pinctrl file routes PA0 to UART TX and PA1 to UART RX using function 9; RX also enables its input path. The custom pinctrl driver writes the generated configuration to the proper PINCM entry at the IOMUX base.

4.2 Dual-base UNICOMM architecture

The UART is part of a configurable UNICOMM block. It has two relevant address regions:

- `0x40632000`: wrapper control for reset, power, and selecting UART mode.
- `0x40601000`: UART clock, control, status, data, baud, and error registers.

Confusing these bases caused an early interrupt experiment to BusFault. The current driver keeps them as separately named devicetree regions.

4.3 Initialization sequence

1. Apply the default pinctrl state.
2. Reset and power the UNICOMM wrapper.
3. Select UART mode.
4. Disable UART while changing configuration.
5. Enable receive/transmit, select 8 data bits/no parity/one stop bit.
6. Select bus clock and divide-by-one.
7. Compute integer/fractional baud divisors with overflow-safe 64-bit intermediates.
8. Latch divisors through the line-control write and enable UART.

At the active 100 MHz source and 115200 baud, the normal 16× oversampling path is appropriate. The driver also contains 8× and 3× cases for baud rates that would otherwise exceed the available clock relationship.

4.4 Polling API

- `poll_in` checks RX FIFO empty; if data exists, it reads the low byte.
- `poll_out` waits while the TX FIFO is full, then writes one byte.
- `err_check` maps overrun, parity, framing, break, and noise bits to Zephyr error flags and clears their latched status.

UART byte transfer remains polling-based. NPU completion is interrupt-driven; these are independent design choices. Polling simplified early board bring-up and is adequate for the single-request protocol, but interrupt or asynchronous UART with buffering is a future production improvement.

5. Protocol-v2 design

5.1 Frame format

Offset	Size	Field	Purpose
0	2	`5a a5`	Magic bytes used to find frame boundaries.
2	1	Version = 2	Reject incompatible wire formats.
3	1	Message type	Request/response meaning.
4	2	Sequence	Match ACK/result/error to a request.
6	2	Payload length	Bound parsing and support multiple message shapes.
8	variable	Payload	Tensor, result, status, or error data.
8 + length	4	IEEE CRC-32	Detect corruption in all fields except magic.

All multibyte values are little-endian, matching the Cortex-M33 and Python `struct` formats. A normal inference request is 268 bytes: 8-byte header + 256-byte tensor + 4-byte CRC.

5.2 Why each field exists

- **Magic:** allows scanning past boot logs, line noise, or damaged bytes.
- **Version:** prevents two incompatible implementations from misinterpreting each other.
- **Type:** supports inference, telemetry, and deliberately gated diagnostics.
- **Sequence:** prevents a delayed response from satisfying the wrong request.
- **Length:** prevents buffer assumptions and validates each message contract.
- **CRC-32:** provides much stronger accidental-corruption detection than the original additive checksum.

5.3 CRC definition

The reflected IEEE CRC-32 uses polynomial `0xedb88320`, initial state `0xffffffff`, and final XOR `0xffffffff`. It covers version, type, sequence, length, and payload. The magic bytes are excluded because they are synchronization markers.

5.4 Receiver behavior

The firmware drains stale bytes, emits `READY`, then applies one absolute one-second deadline to the complete incoming frame. It reports structured errors for version, type, length, CRC, timeout, inference,

internal, and physical UART problems. Test-only NPU/watchdog fault requests are accepted only when their Kconfig options are enabled.

5.5 Why READY and ACK both exist

`READY` means the server is idle and advertises its tensor shape. `ACK` means one particular sequence passed framing/CRC validation and was accepted for a potentially longer action. The host can therefore distinguish "board not ready," "request rejected," and "inference accepted but no result."

6. Python host client

`tools/send_tensor.py` is more than a serial sender. It is the reference implementation of preprocessing and wire validation.

6.1 Responsibilities

- Parse six float vectors and expected outputs from generated C fixtures.
- Parse bias, scale, and shift from the generated model header.
- Quantize and clip exactly 256 values.
- Encode/decode little-endian protocol-v2 frames and CRC.
- Resynchronize after boot text, damaged candidates, or partial frames.
- Require matching message types and sequences.
- Retry transport failures only within a configured budget.
- Reject zero NPU completions or any spurious interrupt.
- Run CRC, NPU stall, watchdog stall, and clock status tests.
- Compute min, mean, median, p95, max, and write machine-readable JSON.

6.2 Sliding resynchronization

The `FrameReader` keeps a byte buffer, searches for magic, waits for the declared candidate size, and validates the candidate. On a bad CRC it discards only one byte and searches again. This matters because the final byte of a damaged frame might actually be the first magic byte of the following valid frame. Dropping the whole candidate could lose recovery data.

7. Generated model and binary ABI

7.1 What the archive exports

- `tvmgen_default_run()` : graph entry point.
- `tvmgen_default_finished` : completion flag.
- `TI_NPU_NNC_IRQHandler()` : generated continuation function.
- Three generated NPU convolution symbols and one average-pool symbol.

- CPU-side generated reshape/pad, cast/add, and dense/postprocessing symbols.

7.2 What the archive requires from the project

- `NPU_getCtlReg()`: address of the TINIElite CTL5 register.
- `NPU_setInstrMem()`: bounded word copies into 1 KiB instruction SRAM.
- `NPU_setParamMem()`: bounded word copies into 2 KiB parameter SRAM.

This interface is an ABI: names, calling convention, types, and data layout must match what the already-compiled archive expects. The copy callbacks return `void`; the integration logs invalid writes but cannot return a failure through this ABI. Eliminating that limitation needs a regenerated compatible runtime or an added out-of-band error mechanism.

7.3 Why not reverse engineer or regenerate it?

The archive is a licensed generated artifact. The original ONNX model, training pipeline, and generator configuration are absent. The responsible approach is to integrity-check and integrate the archive within its license, not claim reproducible training or reconstruct proprietary internals.

8. TINIElite driver and memory interface

8.1 Initialization

The driver follows TI's retained sequence: apply reset control keys, enable power with the required key, wait briefly, grant the Cortex-M33 privileged access through GSC, clear stale DONE status, unmask DONE, connect IRQ 41, clear pending NVIC state, and enable the IRQ.

Hardware “key” fields protect important control registers from accidental writes. Writing only the enable bit would not be accepted.

8.2 Safe SRAM copies

The helper verifies a non-null source, 32-bit-aligned byte offset, nonzero word count, and a range that cannot exceed hardware capacity. It then uses `sys_write32` for each MMIO word. The subtraction-style bound check avoids unsigned addition overflow.

8.3 Why parameters are loaded stage by stage

TINIElite parameter SRAM is only 512 words (2 KiB), so it acts as a staging/scratchpad memory rather than storage for the whole model. An earlier attempt to retain all weights and avoid reloads caused failure. The generated runtime knows what each hardware stage requires and swaps instructions/parameters accordingly.

9. Interrupt-driven inference

9.1 Preparing a run

`tinyengine_prepare_inference()` briefly disables IRQ 41, clears hardware DONE, resets the semaphore, clears pending NVIC state, and re-enables the IRQ. This prevents a stale completion from immediately satisfying a new inference.

9.2 ISR design

The ISR deliberately does very little:

1. Read the masked interrupt status.
2. If DONE is set, write-one-to-clear it.
3. Atomically increment the completion counter.
4. Give the completion semaphore.
5. If DONE is not set, increment the spurious counter.

Generated continuation code is **not** executed inside the ISR. Keeping the ISR short reduces interrupt latency and avoids running opaque, potentially long code in interrupt context.

9.3 Application/runtime handshake

```
prepare interrupt state
start total timer
call tvmgen_default_run()          # generated runtime starts first work

while tvmgen_default_finished == 0:
    wait on semaphore (maximum 100 ms)
    call TI_NPU_NNC_IRQHandler()   # continue graph in thread context

collect elapsed time and IRQ counters
reject zero completions or any spurious completion
```

The same hardware interrupt may represent completion of an internal NPU sublayer. The observed contract is five acknowledged completions per inference. The binary archive does not provide enough source-level visibility to claim an exact one-to-one mapping between the four named NPU functions and all five interrupts.

9.4 Semaphore versus polling

Polling repeatedly reads a register and keeps the CPU runnable. A semaphore lets Zephyr block the thread until the ISR signals progress. This improves schedulability and creates a natural timeout boundary. It does not guarantee lower end-to-end latency; interrupt entry, kernel synchronization, and scheduling add overhead.

10. Timing measurements

The application uses `k_cycle_get_32()` and converts elapsed cycle differences with `k_cyc_to_us_floor32()`. Unsigned subtraction naturally handles one counter wrap as long as the measured interval is shorter than the wrap period.

Metric	What it measures	What it does not mean
`accelerator_us`	Time spent blocked waiting for NPU completion semaphores across stages.	Not pure arithmetic-only silicon time; includes IRQ/wakeup delivery.
`runtime_us`	Time inside initial generated runtime call and continuation calls.	Not total CPU utilization of the whole system.
`total_us`	Wall time from just before runtime start until graph finished.	Does not include UART request/response transfer.

The recovery-enabled image recorded medians of 554 µs accelerator wait, 129 µs runtime, and 701 µs total. The difference between summed components and total includes measurement boundaries and loop/kernel overhead.

11. Timeouts, reset, and watchdog

11.1 NPU-stage timeout

Every hardware stage has a 100 ms semaphore deadline. On timeout, firmware returns structured inference error `-ETIMEDOUT`, waits 10 ms for the final polling-UART byte to leave the transmitter, then requests a cold system reset.

A full reset is intentional. The generated binary may keep continuation globals or partially updated internal state. Without its source-level recovery contract, continuing after a failed stage could produce subtle corruption. Resetting restores a known boot state.

11.2 Why a software timeout is insufficient

The NPU timeout only works if the main application and kernel are still running. A total deadlock, corrupted control flow, or disabled scheduling could prevent that code from executing. Therefore the design also uses an independent hardware watchdog.

11.3 WWDT0 configuration

- Clock: independent 32.768 kHz low-frequency source.
- Configured period: four seconds.
- Feed window: fully open.
- Violation action: BOOTRST rather than NMI.
- Feed policy: only forward progress in the main request loop feeds it.

There is deliberately no background “watchdog feeder” thread. Such a thread might continue feeding while the useful application is deadlocked, hiding the fault the watchdog is meant to catch.

11.4 Fault injection

Test-only protocol requests can deliberately wait for an NPU completion without starting hardware or stop all watchdog feeds. After reboot, the host requires a fresh valid `READY`. For the watchdog test it also checks reset cause `0x0e` and confirms WWDT0 is running again. Kconfig flags are intended to disable these dangerous commands in deployed builds.

12. Application state machine and concurrency

The current design is a single synchronous server, not the original two-thread/message-queue prototype. One main loop:

1. feeds the watchdog;
2. advertises readiness and waits for one request;
3. validates and dispatches clock/watchdog/diagnostic/inference work;
4. runs at most one inference;
5. sends one response; and
6. returns to readiness.

Concurrency still exists at the hardware/kernel boundary: the main thread blocks, the NPU executes independently, and IRQ 41 gives the semaphore. Single-request serialization was chosen because the generated runtime has opaque global state and because it makes ordering, timing, error recovery, and watchdog progress deterministic.

12.1 Buffers and alignment

The application owns static 16-byte-aligned arrays for 256 input bytes and three output bytes. Static storage avoids lifetime errors and dynamic allocation. The main stack is conservatively configured to 32 KiB because the generated runtime uses substantial stack/intermediate space; the total reported RAM footprint includes this reservation.

13. File-by-file responsibility map

File	What to say in an interview
`app/src/main.c`	Synchronous request dispatcher, watchdog progress boundary, inference/error/reset policy.
`app_npu.c`	Generated runtime bridge, interrupt-wait loop, timeout, and timing/IRQ metrics.
`app_uart.c`	Firmware framing, deadlines, CRC validation, structured response encoding.

`app_protocol.c`	Portable CRC-32 and request-contract validation; tested on `native_sim`.
`app_clock.c`	Independent register read-back and clock correctness verdict.
`app_watchdog.c`	Raw WWDT0 setup, progress feed, retained reset-cause reporting, deliberate stall.
`soc/ti/am13x/clock.c`	Pre-kernel crystal/PLL/flash sequence that makes software timing metadata true.
`uart_am13e230x.c`	Dual-base UNICOMM power/configuration and Zephyr polling UART API.
`pinctrl_am13e230x.c`	Applies devicetree-generated PINCM settings through standard Zephyr pinctrl.
`tinyengine_am13e230x.c`	NPU power/privilege, bounded hardware SRAM writes, ISR, counters, semaphore API.
`tools/send_tensor.py`	Reference preprocessing, robust stream parser, regression/stress/fault harness.
`tests/`	Host behavior tests and sanitized target-independent C protocol tests.
`scripts/`	Repeatable WSL developer workflow.

Continue with Volume 3 for the project chronology, failures, corrected measurements, proof matrix, and honest limitations.